Forward Deployed Engineering from First Principles to Enterprise Delivery — Basic

Portfolio projects · lessons 01–09

Each takes 1–3 days and ends with something to show an employer.

Project 1: Basket Bazaar credit path under injected faults

Goal: Prove that the lesson 09 credit path writes each credit exactly once at volume, against a customer API that returns 429s, 503s and lost replies, and show the numbers to an ops lead.

Uses lessons: 02, 03, 04, 05, 07, 09

1. Copy apps/09/mock_erp.py and data/05/orders_clean.csv into your project with the same folder layout (the mock reads parents[2]/data/05/orders_clean.csv), set MOCK_ERP_TOKEN in .env, load it with tut_env.env and print only bool(token), then start uvicorn mock_erp:app --port 8090 from the mock's folder.

2. Write read_all_lines() with the lesson 04 and 09 pager (limit=50, follow next_cursor until it is None, 5 pages) and assert that the line count equals the API's total and the 224 lines in data/05/orders_clean.csv, which hold 120 distinct orders.

3. Write send() with tenacity that retries only 429, 502, 503 and 504 and transport errors, with stop_after_attempt(4), wait_exponential_jitter, and Retry-After honoured when the server sends it.

4. After the read is done, queue 30 faults with POST /lab/faults (10 "429", 10 "503", 10 "lost", shuffled with random.Random(seed)); the 224 credit calls plus 5 read pages make about 229 first calls, so about one call in eight meets a fault before any retry.

5. Write one credit per order line (224 lines), with Idempotency-Key set to the string f"{order_id}|{item}|credit", and send each through send(); then GET /lab/state and assert 224 credits with no repeated (order_id, item) pair.

6. Repeat the whole run with seeds 1, 2 and 3 (POST /lab/reset between runs) and write each run's calls made, retries per fault type, duplicate credits and elapsed seconds to results/run_<seed>.json.

7. Write the lesson 07 one-page status update from the three runs, label every number measured, guess or invented, and rehearse a 5-minute demo that starts from GET /lab/state.

Deliverable: A GitHub repo with README.md (setup in five commands, the run command, a results table with the median and range of duplicate credits, retries per fault type, calls per credit and seconds per run for each seed), the mock, your client code and the one-page status update. Expected: 0 duplicate credits in every run.

Stretch: Queue 5 consecutive "503" faults (a short outage) and show that the client gives up after 4 attempts per credit and marks that case for a person instead of crashing the run. · Compare total seconds per run with and without honouring Retry-After on the 429 replies, and report the measured difference.

Project 2: Basket Bazaar messy-message triage with a measured baseline

Goal: Decide whether a model beats a simple rule for triaging customer delivery messages, using 60 messages you label by hand, a cleaned export and one number per decision.

Uses lessons: 02, 03, 05, 06, 07

1. Write data/messages.csv by hand with 60 invented messages in the lesson 02 columns (message_id, order_id, store, received_at, label, message): 12 each for missing_item, wrong_item, late and damaged, and 12 other (a double charge, a wrong slot). Each order_id must exist in data/05/orders_clean.csv, and you fill every label before any model runs.

2. Add 25 exact copies of rows to make 85 rows, and write 20 of the received_at values as DD/MM/YYYY HH:MM so the file has two date formats; clean it with the lesson 05 steps (trim text, parse both dates, lower-case labels, drop duplicates on the full row) and assert that exactly 25 are removed and 60 rows remain.

3. Build the keyword rule from lesson 06 (one keyword per issue type: missing, wrong, late, damaged; anything else is other) and compute precision and recall per issue type on the 60 labels.

4. Build the model classifier as a Pydantic CaseRecord whose issue is a Literal of the five values, call it through llm_setup.get_llm() under llm_replay (record once with a key in .env, printing only bool() of each value), and run all 60 messages: two model calls per message (a tool lookup, then the form), 120 calls in all.

5. Add the lesson 06 plain-code check against data/05/orders_clean.csv (order_id exists, item is on that order, and a claimed qty is not above the order's qty for each-unit lines) and count how many model outputs it holds back.

6. Score both methods: overall accuracy, per-issue precision and recall, holds, and model calls per message, in one table in results.md.

7. Write a recommendation paragraph that says where the rule is good enough and where the model earns its cost, with the measured numbers quoted.

Deliverable: A GitHub repo with README.md (the labelled messages, the setup, the run command, a results table comparing rule and model per issue type, held outputs and calls per message), the cleaning script and the recording file. Real keys stay in .env and out of the repo; the recording lets the run replay without a key.

Stretch: Write 20 harder messages with two issues in one message or mixed languages, and report how many the model gets fully right versus the rule. · Relabel the 60 messages a week later without looking at the first labels, and report how many labels changed; this is your measured label noise, which caps any score.

Project 3: Freight invoice audit against a shipment-tracking API

Goal: Find carrier billing errors by checking 400 invoice lines against a shipment-tracking API, file one idempotent dispute per error, and report the dollars recovered and the error rate caught, in a logistics domain outside groceries.

Uses lessons: 02, 04, 05, 07, 09

1. Write make_invoices.py with random.Random(7) to create 400 unique invoice lines for 400 shipments over 30 days from 3 invented carriers, each with a billed weight in lb or kg, a rate per kg and a billed amount, and two date formats; add 8 exact copies (408 raw lines), plant 8 billed weights more than 15 percent above tracked weight and 8 invoices dated before delivery, and save the answer key with each planted error's dollar overcharge in answer_key.json.

2. Write a FastAPI mock on port 8092 with GET /shipments (limit 1 to 100, default 25, next_cursor paging), and POST /disputes that keeps the first answer per Idempotency-Key and returns it on a repeat, as lesson 09's mock does; add a 429 with Retry-After: 1 on every third GET call. Its tracked weights and delivery dates come from the same seed.

3. Clean the 408 raw lines with the lesson 05 steps: normalise carrier names, parse both date formats, convert lb to kg at 0.45359237 per lb, and drop duplicates by (carrier, invoice_no, line_no), asserting that 8 are removed and 400 remain.

4. Read all 400 tracked shipments with the lesson 04 pager at limit 100 (4 pages) through a send() that retries 429 and 5xx with tenacity, and assert that the count equals 400.

5. Flag each invoice line as duplicate (the rows removed in step 3), overweight (billed weight above 115 percent of tracked weight) or billed-before-delivery, and compare the flags with answer_key.json for precision and recall per error type.

6. Open one dispute per flagged line with an Idempotency-Key built as f"{carrier}|{invoice_no}|{line_no}|{reason}", with the mock returning 503 on 10 percent of POST calls (seeded); count the retries and the duplicate disputes, which must be zero.

7. Write the lesson 07 status note for the carrier-audit manager with dollars flagged versus dollars planted (from the answer key), the error rate, and every number labelled measured, guess or invented.

Deliverable: A GitHub repo with README.md (the generator and mock, the run command, a results table with precision and recall per error type, dollars flagged versus planted, API calls and retries, duplicate disputes) and the status note. All data is generated; no real carrier or customer data.

Stretch: Add a model step that drafts a one-paragraph dispute letter per flagged line, then a plain-code check that the letter quotes the right invoice_no and amount; report how many letters the check rejects. · Add a rate budget of 60 API calls per minute and measure how many minutes a full audit of 10,000 lines would take with the paging you built.

Project 4: Clinic no-show proof of concept, from discovery to a measured decision

Goal: Turn a vague request to cut no-shows into a two-week proof of concept with a measured kill check on 300 synthetic appointments, and a decision memo a clinic manager can act on.

Uses lessons: 02, 03, 07, 08

1. Write a one-page discovery note from an invented 30-minute call with a fictional clinic manager, sort every line into fact, number or guess (lesson 02), and split the request into 3 problems that each have an observable, a test and an owner.

2. Generate 300 synthetic appointments with random.Random(5) across 3 sites, with columns appointment_id, site, slot_time, lead_days, reminder_sent and no_show (about 18 percent no-show), and no names or contact fields.

3. Measure the baseline no-show rate per site and per reminder_sent value, then write the POC question and 4 success criteria, each with an observable, a baseline, a target, a test and an owner.

4. Write the scope on a 10-working-day sprint with 1 day reserved for the demo, a cut line for work that does not fit, and a non-goals list (no automatic calls, no write-back to the clinic's system).

5. Write the candidate rule down before you score it (lead_days above 7 and no reminder sent), then measure its recall (no-shows it flags, divided by all no-shows) and precision (share of flagged appointments that were no-shows) on the 300 rows; the memo must say the rule was chosen on these same rows, so the score is optimistic.

6. Write the timeline as a CSV with the lesson 08 columns (task, kind, criterion, owner, needs_files, start_day, end_day), a kill check on day 3 (re-measure the baseline), the demo on day 8 and the decision on day 10, and a risk register of 5 risks with the lesson 08 columns (id, risk, likelihood, impact, owner, mitigation, trigger, score), score equal to likelihood times impact (1 to 5 each).

7. Write the decision memo and a 10-minute demo script; label every number measured, guess or invented, and state the kill criterion you would use on day 10.

Deliverable: A GitHub repo with README.md (the discovery note, the generator, the run command, a results table with baseline no-show rate, recall and precision, the timeline CSV, the risk register and the decision memo). Synthetic data only; the README states that no patient data was used.

Stretch: Generate 3 more months of appointments with a changed reminder policy and check whether the rule's recall holds on the new months; report the change in measured numbers. · Write the same status update in two versions, one for the clinic manager and one for the clinic's board, and time how long each takes to read aloud.